وظيفة مقرر الخوارزميات الذكية

BIA601

إعداد الطلاب:

الرقم	اسم الطالب	ID	الصف
1	هشام خليل الشمالي	Hisham_219926	C1
2	عبد الرحيم محمد محفوظ شعراوي	Abdalraheem_190446	C1
3	محمد كنان سعيد حبش	Mohamad_kenan_194865	C4
4	نصري سليم ملكية	Nasri_159174	C4
5	سونيا محمد اياد العبسي الجندي	Sonia_271776	C4
6	عبد العزيز مروان كركرلي	Abdulaziz_249002	C4
7	خالد محمد دياب عبد السلام	Khaled_203913	C5
8	راما عمر الحلاق	Rama_251277	C6

دكتور المقرر

الدكتور عصام محمد سلمان

توزيع المهام على الآلات لتقليل زمن الإنجاز الكلي (Makespan)

تعريف المشكلة:

1- المشكلة:

في العديد من الأماكن مثل المصانع أو الشركات توجد مجموعة من المهام التي يجب توزيعها على عدد من الآلات أو الموظفين ومن المهم أن يكون هذا التوزيع متوازنا لأن توزيع المهام بشكل غير مناسب قد يؤدي إلى أن بعض الآلات تعمل لفترة طويلة بينما تنتهي آلات أخرى من عملها مبكراً وتبقى دون استخدام وهذا قد يسبب تأخيرا في إنهاء جميع المهام.

اخترنا في مشروعنا مشكلة **توزيع المهام على الآلات** وعملنا على إيجاد أفضل طريقة لتوزيع المهام بحيث تنتهي جميعها بأقل وقت ممكن مع محاولة جعل الأحمال موزعة بشكل متوازن بين الآلات.

2-الصياغة الرياضية:

- لدينا n مهمة، وزمن تنفيذ المهمة رقم i هو p_i.
- لدينا m آلة متماثلة (أي أن زمن المهمة نفسه على أي آلة).
- المطلوب: تحديد الآلة التي ستنفذ كل مهمة.
- حمل الآلة j هو مجموع أزمنة المهام الموزعة عليها.
- الهدف هو تقليل Makespan وهو حمل أكثر آلة انشغالا لأن العمل كله لا ينتهي إلا عندما تنتهي هذه الآلة

minimize max_j (sum of p_i for all tasks assigned to machine j)

3-الحد الأدنى النظري:

لا يمكن لأي توزيع أن يعطي Makespan أقل من:

LB = max((sum of p_i) / m , max p_i)

والسبب أن العمل الكلي يجب أن يتوزع على m آلات، فلا يمكن أن يقل أكبر حمل عن متوسط الحمل. وأيضاً فإن أطول مهمة تُنفَّذ كاملة على آلة واحدة، فلا يقل الـ Makespan عن زمنها. استخدمنا هذا الرقم مرجعاً لقياس قرب نتائج الخوارزميات من الحل الأمثل.

4- مثال توضيحي

لدينا 6 مهام بالأزمنة T1=4، T2=7، T3=2، T4=6، T5=3 ، T6=5 و3 آلات، وجرّبنا التوزيع التالي:

الآلة	المهام	الحمل
1	T1,T2	11
2	T3,T4	8
3	T5,T6	8

ال Makespan لهذا التوزيع = 11. مجموع الأزمنة 27 فالحد الأدنى النظري = 27 ÷ 3 = 9 أي أن هذا التوزيع يمكن تحسينه وهذا ما تفعله الخوارزميات.

5- صعوبة المشكلة:

عدد التوزيعات الممكنة يساوي m^n، لأن كل مهمة يمكن وضعها على أي آلة من الآلات. فمثلاً لـ 200 مهمة و10 آلات لا يمكن تجربة كل الاحتمالات. وهذه المشكلة من المشاكل صعبة الحل (NP-hard)، أي لا تُعرف خوارزمية تحلها بزمن معقول لكل الأحجام. لذلك استخدمنا خوارزميات ذكية تبحث عن حل جيد دون فحص كل الحالات.

6-هدف المشروع:

الهدف من مشروعنا هو تصميم وتنفيذ ثلاث خوارزميات ذكية لحل مشكلة توزيع المهام على الآلات، وهي:

- الخوارزمية الجينية (Genetic Algorithm - GA).
- خوارزمية أسراب الجسيمات (Particle Swarm Optimization - PSO).
- خوارزمية مستعمرة النمل (Ant Colony Optimization - ACO).

كما استخدمنا البرمجة الديناميكية (Dynamic Programming - DP) كخط أساس للمقارنة، وذلك للحصول على الحل الأمثل في أحجام المشكلات الصغيرة.

الخوارزميات المستخدمة:

قمنا بكتابة جميع الخوارزميات باستخدام لغة Python، واعتمدنا على مكتبة NumPy فقط في العمليات الحسابية، دون استخدام مكتبات جاهزة لتنفيذ الخوارزميات نفسها.

الخوارزمية الأولى: الخوارزمية الجينية (Genetic Algorithm)

تعتمد الخوارزمية الجينية على فكرة التطور الطبيعي، حيث نبدأ بمجموعة من الحلول العشوائية، ونحاول تحسينها مع مرور الأجيال. في كل جيل نختار الحلول الأفضل ثم ندمج بينها للحصول على حلول جديدة، ونكرر هذه العملية حتى نصل إلى نتائج أفضل.

1- تمثيل الحل (Solution Representation):

مثّلنا الحل بمتجه من الأعداد الصحيحة طوله n، حيث assign[i] = j تعني أن المهمة i تنفَذ على الآلة j. مثلا المتجه [0, 2, 1, 1, 0] يعني أن المهمتين الأولى والخامسة على الآلة 0، والمهمتين الثالثة والرابعة على الآلة 1، والمهمة الثانية على الآلة 2. هذا التمثيل مباشر، وأي متجه بهذا الشكل هو حل صالح دائماً.

2- دالة الهدف (Fitness):

دالة الهدف هي الـ Makespan، أي حمل أكثر آلة انشغالا وكلما قلت كان الحل أفضل.

3- آلية عمل الخوارزمية:

1. **التهيئة:** نولد 100 حل عشوائي.
2. **الاختيار (Tournament Selection):** لكل أب نختار 3 حلول عشوائياً ونأخذ الأفضل بينها.
3. **التزاوج (Uniform Crossover):** نأخذ أبوين، ولكل مهمة نختار آلتها من الأب الأول أو الثاني باحتمال 50%، وذلك باحتمال تزاوج 0.9.
4. **الطفرة (Mutation):**كل مهمة تتغير آلتها إلى آلة عشوائية باحتمال "(n/1)" لنحافظ على التنوع ولا نعلق في حل محلي.
5. **النخبوية (Elitism):** ننقل أفضل حلّين للجيل التالي كما هما حتى لا نفقد أفضل ما وجدنا.
6. نكرر الخطوات من 2 إلى 5 لكل جيل.

4- معيار التوقف:

تتوقف الخوارزمية عند استهلاك ميزانية التقييم المحددة لكل حجم (10,000 و30,000 و60,000 تقييم). يوجد في الكود خيار للتوقف المبكر عند عدم التحسن (patience)، لكنه غير مفعّل في التجارب.

5- المعاملات (Hyperparameters):

المعامل	القيمة	المعنى
حجم المجتمع	100	عدد الحلول في كل جيل
احتمال التزاوج pc	0.9	احتمال مزج الأبوين
احتمال الطفرة pm	1/n	احتمال تغيير آلة كل مهمة
حجم Tournament	3	عدد المتنافسين في الاختيار
Elitism	2	عدد الحلول المحفوظة من الجيل السابق

الخوارزمية الثانية: خوارزمية أسراب الجسيمات (PSO)

تعتمد خوارزمية PSO على محاكاة حركة أسراب الطيور والأسماك. في هذه الخوارزمية لدينا مجموعة من الجسيمات وكل جسيم يمثل حلا محتملا للمشكلة. تتحرك الجسيمات داخل فضاء البحث، ويحتفظ كل جسيم بأفضل موقع وصل إليه سابقا كما يستفيد من أفضل موقع وصل إليه السرب بالكامل. ومن خلال الجمع بين هذين العاملين تحاول الجسيمات الوصول إلى حلول أفضل.

1- تمثيل الحل (Solution Representation):

كل جسيم هو متجه من الأعداد الحقيقية طوله n وكل قيمة فيه بين 0 وm. لتحويله إلى توزيع فعلي نأخذ الجزء الصحيح من كل قيمة: assign[i] = floor(x[i]) مثلا إذا كان m = 3 فالقيمة 1.7 تعني الآلة 1 والقيمة 2.3 تعني الآلة 2. القيم محصورة ضمن المدى بحيث لا نحصل على آلة غير موجودة.

2- دالة الهدف (Fitness):

نفس دالة الهدف: الـ Makespan بعد تحويل الجسيم إلى توزيع، وكلما قل كان أفضل.

3- آلية عمل الخوارزمية:

1. **التهيئة:** نولّد 100 حل عشوائي.
2. **التقييم:** نحوّل كل جسيم إلى توزيع ونحسب الـ Makespan.
3. **تسجيل الأفضل:** كل جسيم يحفظ أفضل موقع له (pbest) والسرب يحفظ أفضل موقع عام (gbest).
4. **تحديث السرعة:**

$$v = w \cdot v + c1 \cdot r1 \cdot (pbest - x) + c2 \cdot r2 \cdot (gbest - x)$$

   - الحد الأول (القصور w): يحافظ على اتجاه الحركة السابق.
   - الحد الثاني (c1): يشد الجسيم نحو أفضل موقع له.
   - الحد الثالث (c2): يشد الجسيم نحو أفضل موقع للسرب.
   - r1 وr2 أرقام عشوائية بين 0 و1.
5. **تحديث الموقع:** x = x + v مع حصر السرعة والموقع ضمن حدود فضاء البحث.
6. نكرر الخطوات من 2 إلى 5 لكل تكرار.

وقيمة w تتناقص خطيا من 0.9 إلى 0.4: بالبداية حركة واسعة لاستكشاف الفضاء وبالنهاية حركة أدق حول الحل الجيد.

4- معيار التوقف:

نفس معيار GA: استهلاك ميزانية التقييم (10,000 و30,000 و60,000). خيار patience موجود بالكود لكنه غير مفعل.

5- المعاملات (Hyperparameters):

المعامل	القيمة	المعنى
حجم السرب	100	عدد الجسيمات
w (القصور)	من 0.9 إلى 0.4	يتناقص خطياً مع التكرارات
c1	2.0	الانجذاب لأفضل موقع شخصي
c2	2.0	الانجذاب لأفضل موقع للسرب
أقصى سرعة	0.5 × m	يمنع القفزات الكبيرة

الخوارزمية الثالثة: خوارزمية مستعمرة النمل (Ant Colony Optimization)

تعتمد خوارزمية ACO على محاكاة طريقة بحث النمل عن الطعام. فعندما يتحرك النمل في طريق معين يترك خلفه مادة تسمى الفيرومون وتستخدم بقية النملات هذه المادة للاستدلال على الطرق التي سلكتها النملات السابقة ومع مرور الوقت تزداد كمية الفيرومون على الطرق التي تؤدي إلى نتائج أفضل وبالتالي تصبح هذه الطرق أكثر احتمالاً للاختيار.

في مشروعنا اعتبرنا أن اختيار وضع مهمة معينة على آلة محددة يمثل جزءاً من الطريق الذي تبنيه النملة.

1- تمثيل الحل (Solution Representation):

الحل النهائي هو نفسه في GA: المتجه assign[i] = j. الفرق أن النملة **تبني** الحل خطوة خطوة فتضع مهمة واحدة في كل خطوة :
إضافة إلى ذلك لدينا مصفوفة فيرومون τ[i][j] بحجم n × m وكل خانة فيها تعبر عن مدى تفضيل وضع المهمة i على الآلة j .

2- دالة الهدف (Fitness):

نفس الدالة: الـ Makespan، وكلما قلّ كان الحل أفضل.

3- آلية عمل الخوارزمية:

1. **التهيئة:** مصفوفة الفيرومون كلها قيمتها 1.
2. **بناء الحلول:** كل نملة (من 50) توزع المهام واحدة واحدة وترتيب المهام من الأطول للأقصر (لأن وضع المهام الكبيرة أولا يعطي توازناً أفضل) ولكل مهمة تختار النملة آلة بالاحتمال التالي:

$$P(j) \propto \tau[i][j]^{\alpha} \times \eta[j]^{\beta}$$

$$\eta[j] = 1 / (load[j] + p[i])$$

حيث load[j] هو حمل الآلة j الحالي، و p[i] هو زمن المهمة i.
3. **التقييم:** نحسب الـ Makespan لكل نملة.
4. **التبخر:** نضرب كل الفيرومون بـ (1 − ρ) حتى ننسى الطرق القديمة ولا نعلق بحل سيء.
5. **الترسيب:** أفضل 3 نملات بالتكرار تترك فيرومون على الخيارات التي اتخذتها بمقدار Q / Makespan (الحل الأقصر يترك أكثر). وأيضا أفضل حل وجد حتى الآن يعزز نفسه كل تكرار.
6. **حصر الفيرومون:** نبقيه بين 0.01 و10 حتى لا يصبح أي احتمال صفرا ولا تسيطر آلة معينة مبكرا.
7. نكرر الخطوات من 2 إلى 6 لكل تكرار.

4- معيار التوقف:

نفس باقي الخوارزميات: استهلاك ميزانية التقييم. عدد التكرارات = الميزانية ÷ عدد النمل (50) فمثلا بالحجم الكبير

60,000 ÷ 50 = 1200 تكرار. خيار patience موجود بالكود وغير مفعل.

5- المعاملات (Hyperparameters):

المعامل	القيمة	المعنى
عدد النمل	50	عدد الحلول المبنية في كل تكرار
α	1.0	أهمية الفيرومون
β	2.0	أهمية معلومة «الآلة الأقل حملا»
ρ (التبخر)	0.1	نسبة الفيرومون التي تختفي في كل تكرار
Q	مجموع الأزمنة ÷ m	ثابت كمية الترسيب
عدد النمل المرسِّب	3	أفضل نملات التكرار
حدود الفيرومون	0.01 إلى 10	يمنع الجمود المبكر

البرمجة الديناميكية (Dynamic Programming - DP):

استخدمنا DP كخط أساس (Baseline) لأنها تعطي **الحل الأمثل الحقيقي** فنستطيع قياس مدى قرب الخوارزميات الذكية من الأمثل.

الحالة (f[k][mask]) هي أقل Makespan ممكن عند توزيع مجموعة المهام mask على أول k آلات. وللانتقال من k−1 آلة إلى k آلة نجرب كل مجموعة جزئية s من المهام نعطيها للآلة الجديدة:

f[k][mask] = min over s subset of mask of max(f[k-1][mask - s] , sum(s))

1-التعقيد والحدود:

يبلغ التعقيد الزمني لهذه الخوارزمية:

$$[O(m \cdot 3^n)]$$

وهذا يجعل استخدامها مناسباً للأحجام الصغيرة فقط، لأن عدد الحالات يزداد بسرعة كبيرة مع زيادة عدد المهام.

في مشروعنا، استغرق حل الحجم الصغير الذي يحتوي على 12 مهمة حوالي ثانية واحدة. أما الحجم المتوسط الذي يحتوي على 50 مهمة والحجم الكبير الذي يحتوي على 200 مهمة فلم يكن من الممكن حلهما باستخدام هذه الطريقة.

لذلك استخدمنا الحد الأدنى النظري (LB) كمرجع للمقارنة في الحجمين المتوسط والكبير.

2-النتيجة:

أعطت DP القيمة المثلى **196** للحجم الصغير وهي تساوي الحد الأدنى النظري أي أن 196 هي أفضل قيمة ممكنة فعلا.

إعداد التجارب (Experimental Setup)

1- أحجام المشكلة:

اختبرنا الخوارزميات على ثلاثة أحجام:

الحجم	عدد المهام (n)	عدد الآلات (m)	الحد الأدنى النظري (LB)
صغير Small	12	3	196.0
متوسط Medium	50	5	546.4
كبير Large	200	10	1027.7

أزمنة المهام أعداد صحيحة عشوائية بين 1 و100، ولُوِّدت ببذرة ثابتة (seed = 42) حتى تكون المسألة نفسها في كل تشغيل.

2- عدالة المقارنة (ميزانية التقييم):

تختلف الخوارزميات في حجم المجتمع (GA وPSO: 100، وACO: 50 نملة) فلو قارنا بعدد التكرارات لكانت المقارنة غير عادلة. لذلك أعطينا كل الخوارزميات **نفس ميزانية التقييم** أي نفس عدد المرات التي تُحسب فيها قيمة الـ Makespan لكل حجم:

الحجم	ميزانية التقييم	تكرارات GA وPSO	تكرارات ACO
صغير	10,000	حوالي 100	200
متوسط	30,000	حوالي 300	600
كبير	60,000	حوالي 600	1200

ولذلك يختلف عدد التكرارات على محور x في منحنيات التقارب بين الخوارزميات مع أن العمل الحسابي الكلي متساو.

3- تكرار التجارب:

شغلنا كل خوارزمية **10 مرات** على كل حجم ببذور عشوائية مختلفة (0 إلى 9) حتى نقيس **استقرار** النتائج أي كم تتغير النتيجة من تشغيل لآخر المجموع: 3 خوارزميات × 3 أحجام × 10 تشغيلات = 90 تشغيلا إضافة إلى حل DP للحجم الصغير.

4- المقاييس:

- **جودة الحل:** الـ Makespan النهائي (المتوسط والانحراف المعياري وأفضل وأسوأ قيمة).
- **زمن التنفيذ:** بالدالة time.perf_counter() بالثواني.
- **استهلاك الذاكرة:** بالمكتبة tracemalloc وهي تقدير نسبي للمقارنة وليست الذاكرة الكاملة للجهاز.
- **سرعة التقارب:** منحنى التكرار مقابل أفضل قيمة (Iteration → Best Fitness) متوسط 10 تشغيلات.
- **الاستقرار:** الانحراف المعياري وصناديق الـ Boxplot.

5- البيئة والأدوات:

كتبنا الخوارزميات الأربع يدوياً بلغة Python مستخدمين مكتبة NumPy فقط (بدون مكتبات جاهزة للخوارزميات). واستخدمنا pandas لحفظ النتائج وmatplotlib للرسوم.

6- اختبارات الصحة والتكرار:

- أعدنا التجارب على نسخة نظيفة من المشروع (بيئة افتراضية جديدة وتثبيت المكتبات من requirements.txt) فتطابقت نتائج الجودة تماما بينما اختلفت الأزمنة قليلا حسب حالة الجهاز.
- استخدمنا الملف sanity_check.py لإجراء مجموعة من الاختبارات وكانت جميعها ناجحة. وشملت هذه الاختبارات التأكد من أن كل مهمة تم إسنادها إلى آلة صحيحة وأن قيمة Makespan المحسوبة تطابق القيمة التي يتم إعادة حسابها من التوزيع وأنه لا توجد نتيجة أقل من الحد الأدنى النظري (LB).
- تحققنا من أن منحنى التقارب لا تزداد قيمته مع التكرارات وأن استخدام نفس البذرة العشوائية يعطي النتيجة نفسها. وبالنسبة إلى خوارزمية DP تمت مقارنة نتائجها مع طريقة Brute Force على مثال صغير للتأكد من صحة الحل.

النتائج:

يوضح الجدول التالي نتائج الـ Makespan لكل خوارزمية بعد تشغيلها 10 مرات. ويعرض الجدول متوسط النتائج والانحراف المعياري، بالإضافة إلى أفضل وأسوأ نتيجة تم الوصول إليها.

الحجم	الخوارزمية	المتوسط	الانحراف المعياري	الأفضل	الأسوأ	الأمثل / LB
صغير	GA	197.2	0.42	197	198	196 (DP)
صغير	PSO	197.3	1.16	196	200	196 (DP)
صغير	ACO	**196.6**	0.70	196	198	196 (DP)
متوسط	GA	**548.9**	1.97	547	554	546.4
متوسط	PSO	553.7	2.36	550	556	546.4
متوسط	ACO	551.4	1.90	548	555	546.4
كبير	GA	**1031.7**	2.26	1030	1037	1027.7
كبير	PSO	1081.6	10.00	1058	1092	1027.7
كبير	ACO	1051.0	4.27	1045	1058	1027.7

ولتوضيح قرب النتائج من الحل حسبنا نسبة زيادة متوسط كل خوارزمية عن الأمثل (الحجم الصغير) أو عن الحد الأدنى النظري (المتوسط والكبير):

الحجم	GA	PSO	ACO
صغير	0.61%	0.66%	0.31%
متوسط	0.46%	1.34%	0.92%
كبير	0.39%	5.24%	2.27%

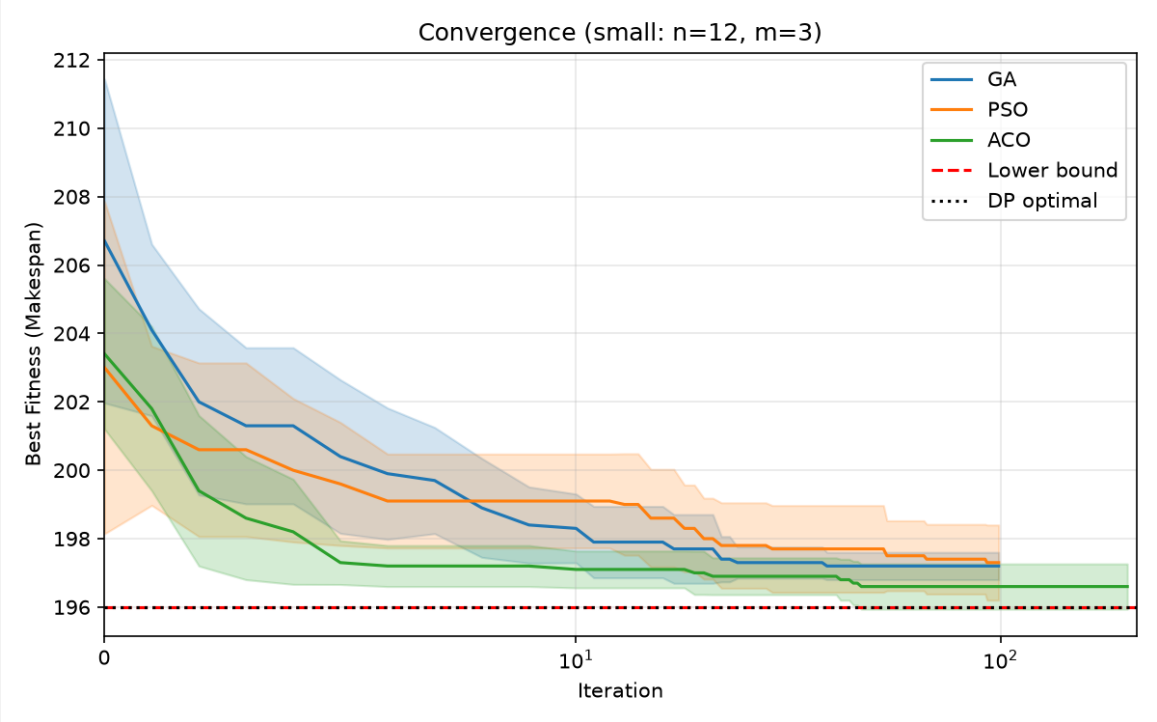
ونلاحظ من النتائج ما يأتي:

- في الحجم الصغير كانت ACO الأفضل بمتوسط 196.6 ووصلت إلى الحل الأمثل 196 في 5 تشغيلات من 10. ووصلت PSO إليه في تشغيلين فقط، أما GA فلم تصل إليه في أي تشغيل (أفضل قيمة لها 197).
- في الحجمين المتوسط والكبير كانت GA الأفضل وكانت قريبة جداً من الحد الأدنى النظري (أقل من 0.5%).
- كانت PSO الأضعف في الحجمين المتوسط والكبير وزادت الفجوة مع زيادة حجم المشكلة (5.24% في الحجم الكبير).
- كل النتائج أكبر من أو تساوي الحد الأدنى النظري وهذا متوقع ويؤكد سلامة حساب الـ Makespan.

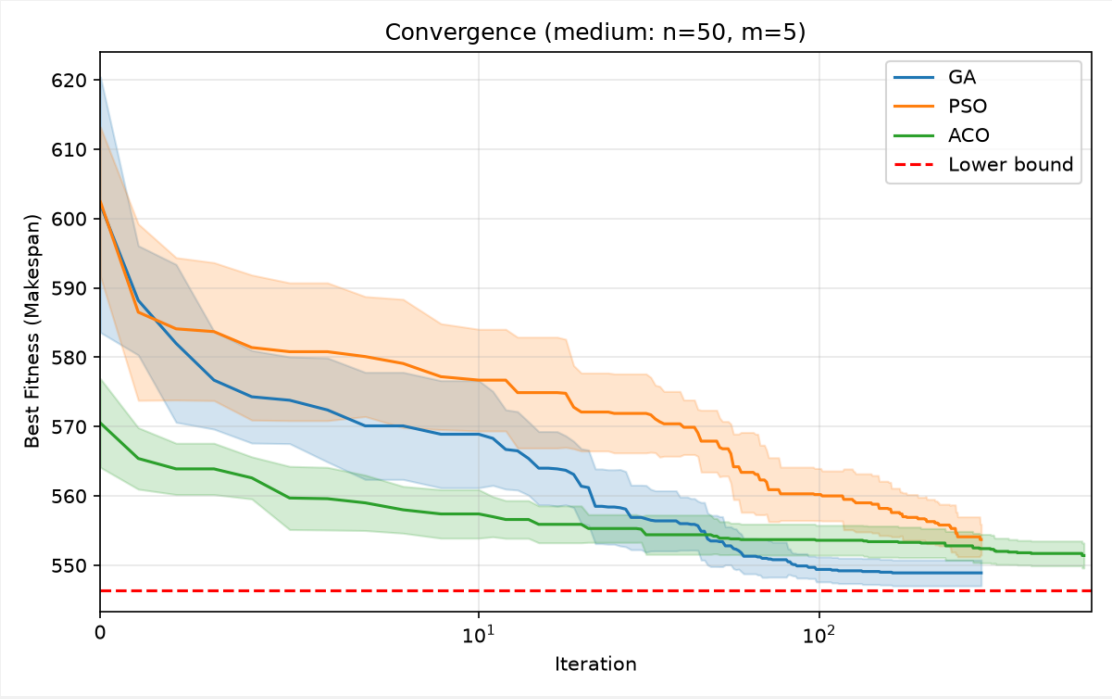
1- سرعة التقارب (Convergence):

توضح الأشكال من 1 إلى 3 منحنيات التقارب للخوارزميات في أحجام المسائل الثلاثة. ويمثل كل منحنى متوسط أفضل قيمة تم الوصول إليها خلال 10 تشغيلات بينما توضح المنطقة المظللة حول المنحنى مقدار الانحراف المعياري.

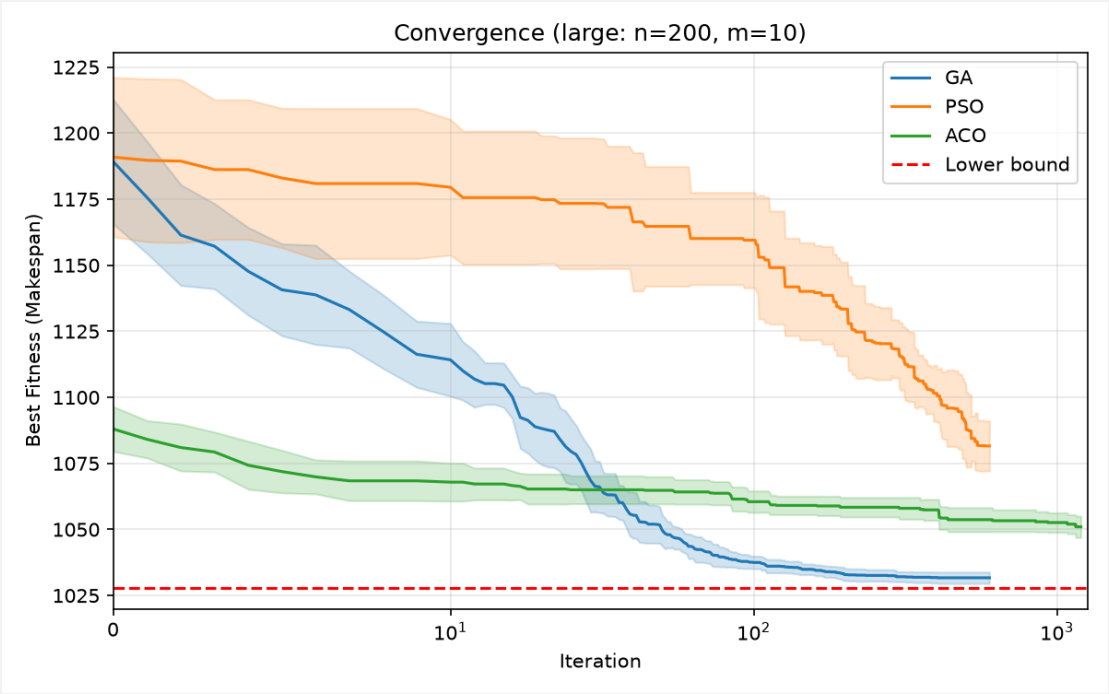
الخط الأحمر المتقطع يمثل الحد الأدنى النظري أما الخط الأسود المنقط في حالة الحجم الصغير فيمثل الحل الأمثل الذي حصلنا عليه باستخدام DP. واستخدمنا مقياسا لوغاريتميا لمحور التكرارات كما أن عدد التكرارات يختلف بين الخوارزميات بسبب اختلاف أحجام المجتمعات رغم أن ميزانية التقييم موحدة.



الشكل 1: منحنى التقارب للحجم الصغير (12 مهمة و3 آلات)



الشكل 2: منحنى التقارب للحجم المتوسط (50 مهمة و5 آلات)



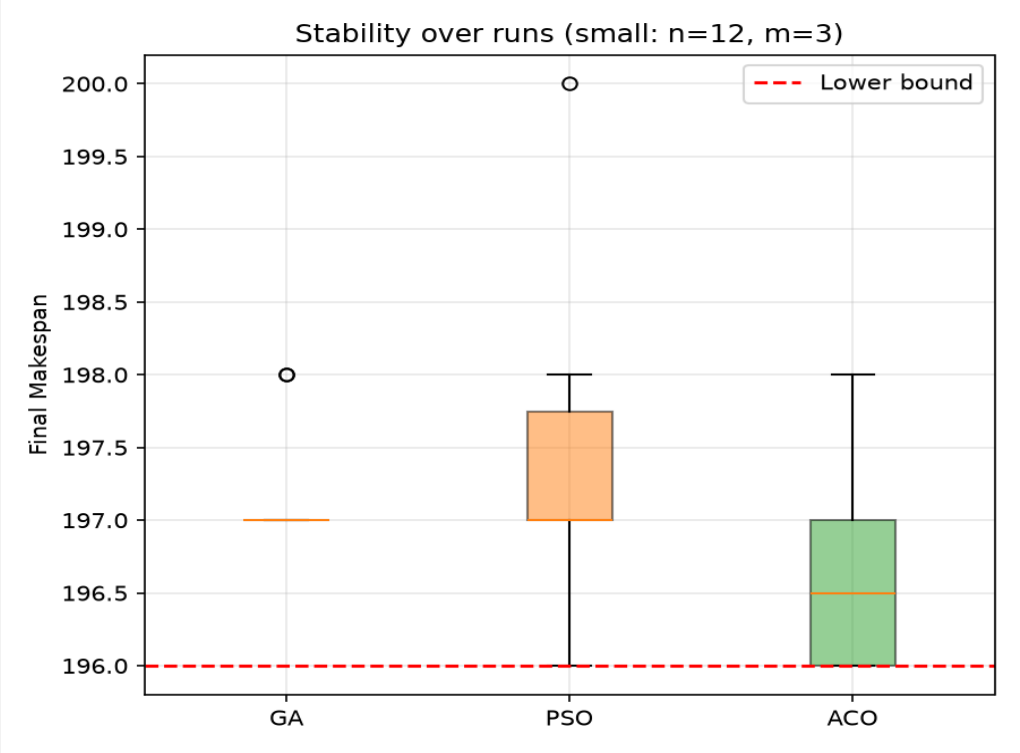
الشكل 3: منحنى التقارب للحجم الكبير (200 مهمة و10 آلات)

في الحجم الكبير نلاحظ أن ACO بدأت بقيمة أفضل من GA وPSO، حيث كانت قيمة البداية حوالي 1085 مقابل حوالي 1190

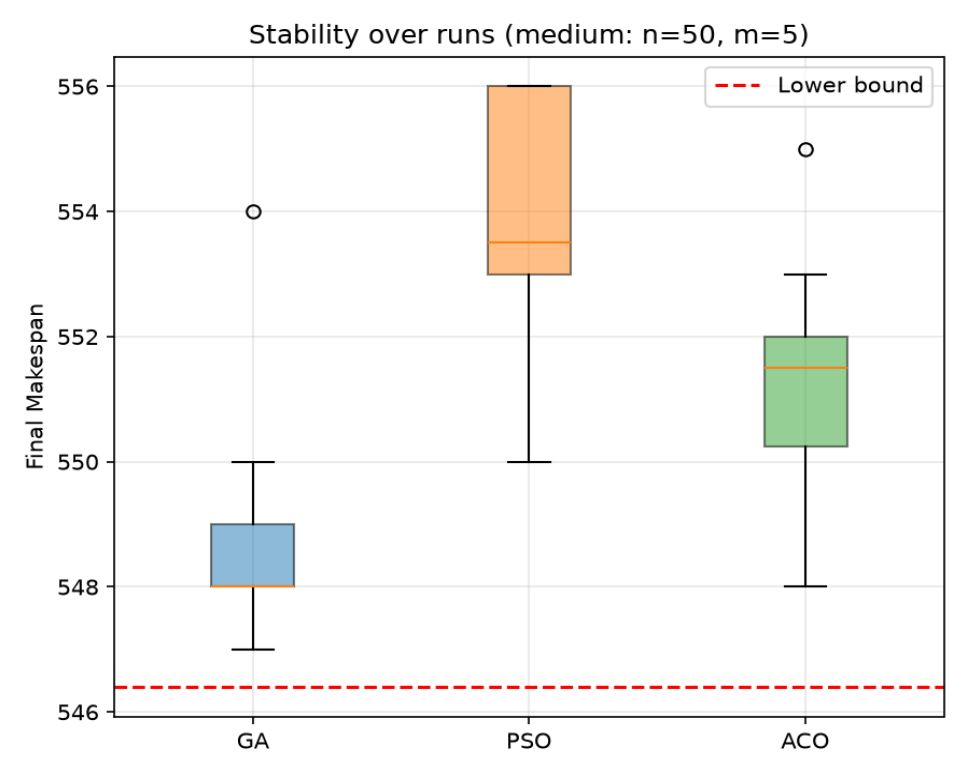
لكن التحسن في ACO أصبح أبطأ مع تقدم التكرارات. بدأت GA بنتيجة أضعف لكنها تحسنت بسرعة واستطاعت تجاوز ACO ثم استقرت عند قيمة قريبة من الحد الأدنى النظري. أما PSO فكان تحسنها أبطأ من الخوارزميات الأخرى واستمر منحناها بالانخفاض حتى نهاية ميزانية التقييمات وهذا يدل على أنها كانت لا تزال تحاول تحسين الحل

2- استقرار النتائج (Stability):

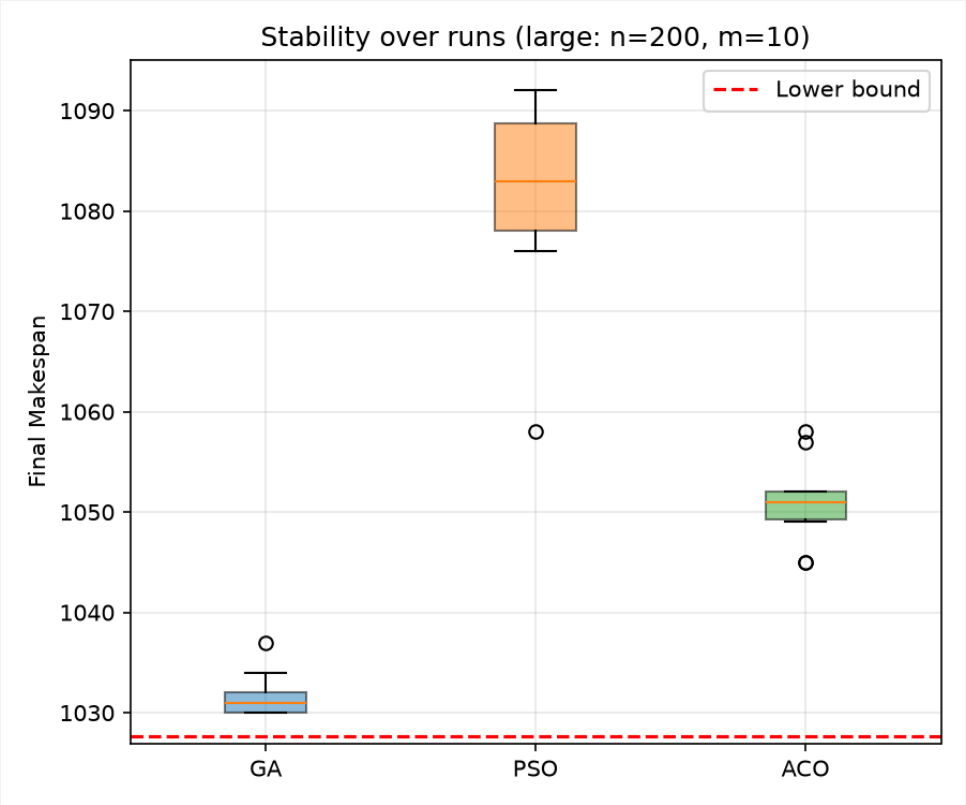
توضح الأشكال من 4 إلى 6 توزيع النتائج النهائية التي حصلنا عليها خلال التشغيلات العشرة لكل خوارزمية. ومن خلال مخططات الصندوق (Boxplot) يمكن معرفة مدى استقرار النتائج فكلما كان الصندوق أضيق كان التشتت أقل وبالتالي تكون الخوارزمية أكثر استقرارا أما النقاط الدائرية التي تظهر خارج الصناديق، فهي تمثل القيم الشاذة (Outliers) أي النتائج التي تختلف عن بقية التشغيلات.



الشكل 4: استقرار النتائج في الحجم الصغير



الشكل 5: استقرار النتائج في الحجم المتوسط



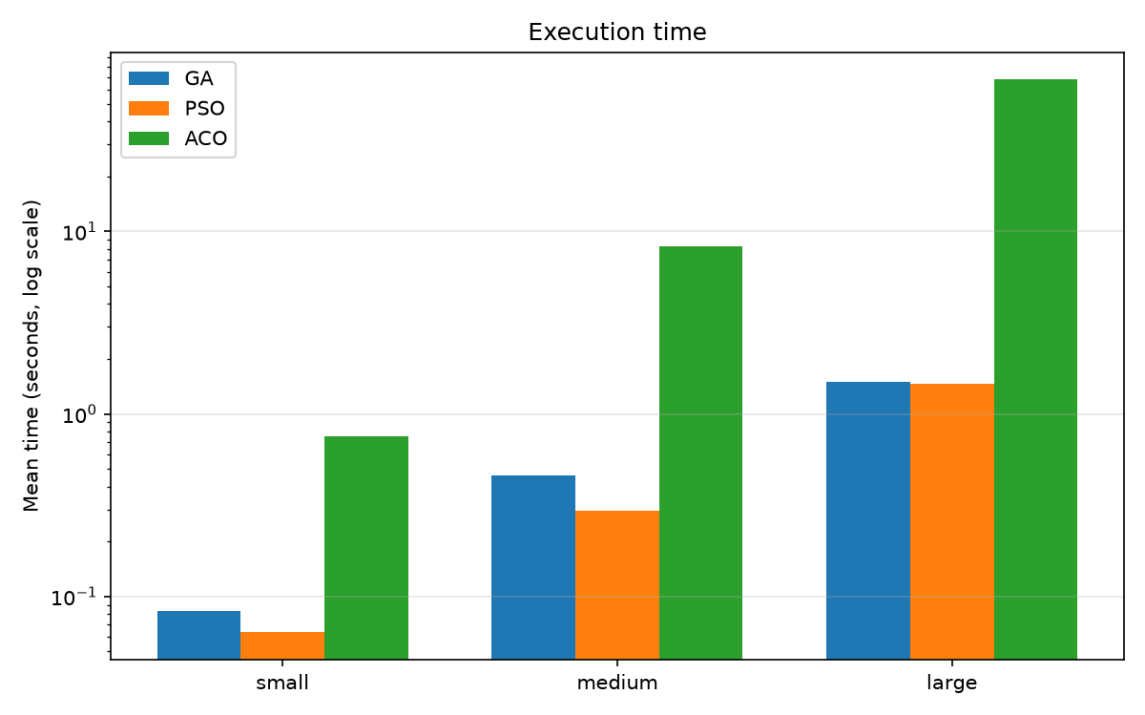
الشكل 6: استقرار النتائج في الحجم الكبير

من النتائج نلاحظ أن GA كانت الأكثر استقرارا في الحجمين المتوسط والكبير حيث بلغ الانحراف المعياري 1.97 و2.26 على التوالي.

أما PSO فكانت الأقل استقرارا خصوصاً في الحجم الكبير حيث وصل الانحراف المعياري إلى 10.00 وتراوحت نتائجها بين 1058 و1092. وبالنسبة إلى ACO فقد كان استقرارها متوسطا مقارنة بالخوارزميتين وبلغ الانحراف المعياري لها 4.27 في الحجم الكبير.

3- زمن التنفيذ (Execution Time):

الحجم	GA (ثانية)	PSO (ثانية)	ACO (ثانية)
صغير	0.08	0.06	0.76
متوسط	0.46	0.29	8.31
كبير	1.49	1.46	67.79



الشكل 7: متوسط زمن التنفيذ لكل خوارزمية (مقياس لوغاريتمي)

نلاحظ أن GA و PSOكانتا متقاربتين من ناحية زمن التنفيذ وكلتاهما أسرع بكثير من .ACO ففي الحجم الكبير احتاجت ACO إلى حوالي 68 ثانية، بينما احتاجت GA إلى حوالي 1.5 ثانية فقط أي أن ACO كانت أبطأ بحوالي **45 مرة**.

4- استهلاك الذاكرة:

الحجم	GA (KB)	PSO (KB)	ACO (KB)
صغير	79	95	**26**
متوسط	306	369	**63**
كبير	1029	1419	**211**

استهلكت ACO أقل ذاكرة واستهلكت PSO أكثر ذاكرة. ويمكن تفسير ذلك بأن PSO تحتفظ بعدة مصفوفات كبيرة لمجتمع من 100 جسيم (المواقع والسرعات وأفضل المواقع والأعداد العشوائية) بينما تعمل ACO بمجموعة من 50 نملة فقط وهذه القيم تقدير نسبي لذاكرة Python.

المناقشة:

اختلف أداء الخوارزميات حسب حجم المسألة وطبيعة كل خوارزمية ويمكن تلخيص نقاط القوة والضعف كالتالي:

- **الخوارزمية الجينية (GA):** حققت أفضل توازن بين جودة الحل وسرعة التنفيذ في الحجمين المتوسط والكبير وكانت الأكثر استقرارا ويعود ذلك إلى أن التمثيل المباشر (آلة لكل مهمة) مناسب للتزاوج والطفرة إذ يحافظ الأبناء على أجزاء جيدة من الأبوين. أما ضعفها فهو بدايتها من حلول عشوائية، فتحتاج أجيالاً عدة لتصل إلى مستوى ACO في البداية.
- **خوارزمية مستعمرة النمل (ACO):** كانت الأفضل في الحجم الصغير ووصلت إلى الحل الأمثل في نصف التشغيلات وبدأت من قيم أفضل بكثير من GA وPSO بفضل ترتيب المهام من الأطول للأقصر واستخدام معلومة الحمل الحالي. أما ضعفها فهو البطء الكبير، لأن كل نملة تبني حلها مهمة بعد مهمة وتباطؤ التحسن في الحجم الكبير.
- **خوارزمية أسراب الجسيمات (PSO):** سريعة وبسيطة، لكنها الأقل استقرارا والأضعف في الحجمين المتوسط والكبير. ويحتمل أن السبب أن الحركة في فضاء مستمر ثم تحويلها بـ floor إلى رقم آلة لا تحافظ على معنى القرب بين الحلول، فجسيمان قريبان قد يمثلان توزيعين مختلفين تماماً. كما لم نضبط معاملاتها بشكل خاص وكان منحناها ما زال ينخفض عند نهاية الميزانية.

البرمجة الديناميكية (DP): تعطي الحل الأمثل فعلا لكنها لا تصلح إلا للأحجام الصغيرة بسبب تعقيدها $[O(m \cdot 3^n)]$

لذلك يعتمد اختيار الخوارزمية الأنسب على حجم المشكلة والأولوية بين جودة الحل وسرعة التنفيذ.

الخاتمة:

درسنا في هذا المشروع مشكلة توزيع المهام على الآلات لتقليل زمن الإنجاز الكلي ونفذنا ثلاث خوارزميات ذكية (GA وPSO وACO) من الصفر، وقارناها مع الحل الأمثل بالبرمجة الديناميكية على ثلاثة أحجام. وأهم ما توصلنا إليه:

- الخوارزمية الجينية هي الأفضل عموما في الحجمين المتوسط والكبير وهي الأكثر استقرارا وتجمع بين جودة جيدة وزمن قصير.
- خوارزمية مستعمرة النمل هي الأفضل في الحجم الصغير ووصلت إلى الحل الأمثل (196) في 5 تشغيلات من 10، لكنها الأبطأ بفارق كبير.
- خوارزمية أسراب الجسيمات هي الأضعف في هذه المسألة بسبب تحويل المتجه الحقيقي إلى أرقام آلات وتزداد فجوتها مع كبر الحجم.
- البرمجة الديناميكية أكدت أن الحل الأمثل للحجم الصغير هو 196 لكنها لا تصلح للأحجام الكبيرة.

وتعلمنا أن اختيار الخوارزمية يعتمد على طبيعة المسألة وطريقة تمثيل الحل، وأن الخوارزمية التي تناسب مسألة معينة قد لا تناسب غيرها.

ملحق: طريقة تشغيل المشروع:

هيكل المشروع: المجلد src فيه ملف المسألة problem.py وملفات التجارب والرسوم والاختبارات، والمجلد src/algorithms فيه الخوارزميات الأربع، والمجلد results فيه النتائج والرسوم. وللتشغيل من المجلد الرئيسي:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux / Mac
pip install -r requirements.txt
python src/experiments.py
python src/plots.py
python src/sanity_check.py
```

مساهمة أعضاء الفريق:

الرقم	الصف	اسم الطالب	ID	المهمة
1	C1	هشام خليل الشمالي	Hisham_219926	تعريف المشكلة وكتابة ملف problem.py
2	C1	عبد الرحيم محمد محفوظ شعراوي	Abdalraheem_190446	تنفيذ الخوارزمية الجينية GA
3	C4	محمد كنان سعيد حبش	Mohamad_kenan_194865	تنفيذ خوارزمية أسراب الجسيمات PSO
4	C4	نصري سليم ملكية	Nasri_159174	تنفيذ خوارزمية مستعمرة النمل ACO
5	C4	سونيا محمد اياد العبسي الجندي	Sonia_271776	تنفيذ البرمجة الديناميكية (DP) واختبارات الصحة
6	C4	عبد العزيز مروان كركرلي	Abdulaziz_249002	كتابة ملف التجارب (experiments.py) وتشغيلها
7	C5	خالد محمد دياب عبد السلام	Khaled_203913	الرسوم البيانية وتحليل النتائج plots.py
8	C6	راما عمر الحلاق	Rama_251277	كتابة التقرير وملف README والمراجعة